Reattempt Container Orchestration

git clone https://github.com/mohanDevOps-arch/Blue-green-Deployment.git
cd blue-green-app

Install Dependency

sudo apt install npm

cd backend
npm install

cd ../frontend-basic
npm install

cd ../frontend-enhanced
npm install

MongoDB Setup (Docker Mongo)

sudo apt install -y docker.io
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker $USER

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ sudo apt install docker.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools cgroupfs-mount | cgroup-lite debootstrap docker-buildx docker-compose-v2 docker-d
The following NEW packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base docker.io pigz runc ubuntu-fan
0 upgraded, 8 newly installed, 0 to remove and 83 not upgraded.
Need to get 76.1 MB of archives.
After this operation, 288 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
```

docker run -d --name mongodb -p 27017:27017 mongo:6

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ sudo docker run -d \
  --name mongodb \
  -p 27017:27017 \
  mongo:6
Unable to find image 'mongo:6' locally
6: Pulling from library/mongo
7e49dc6156b0: Pull complete
5ca411ad8051: Pull complete
baf1f36682ef: Pull complete
3c99b8c985d2: Pull complete
a44afe9952bf: Pull complete
9826927a661f: Pull complete
7b14d5e3f398: Pull complete
e73078607ea4: Pull complete
Digest: sha256:03cda579c8caad6573cb98c2b3d5ff5ead452a6450561129b89595b4b9c18de2
Status: Downloaded newer image for mongo:6
13dcd5f39dd04f5b3ccf71438b1686ef39d9dc4e959a4c645046a6098771c7c4
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$
```

Backend .env Configuration

backend/.env

PORT=5000
MONGO_URI=mongodb://localhost:27017/bluegreen

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ cd backend/
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/backend$ sudo nano .env
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/backend$ cat .env
PORT=5000
MONGO_URI=mongodb://localhost:27017/bluegreen
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/backend$
```

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/backend$ npm start

> registration-backend@1.0.0 start
> node server.js

Backend server running on port 5000
MongoDB connected
^C
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/backend$
```

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ cd frontend-green/
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/frontend-green$ npm start

> green-frontend@1.0.0 start
> node server.js

Green frontend server running on port 3200
^C
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/frontend-green$
```

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ cd frontend-blue/
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/frontend-blue$ npm start

> basic-frontend@1.0.0 start
> node server.js

Basic frontend server running on port 3100
Accessible at http://localhost:3100
```

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ cd backend/
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/backend$ sudo nano Dockerfile
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/backend$ cat Dockerfile
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --production

COPY . .

EXPOSE 5000

CMD ["node", "server.js"]
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/backend$
```

FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --production

COPY . .

EXPOSE 5000

CMD ["node", "server.js"]

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ sudo apt install docker-compose
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  python3-compose python3-docker python3-dockerpty python3-docopt python3-dotenv python3-texttable python3-websocket
The following NEW packages will be installed:
  docker-compose python3-compose python3-docker python3-dockerpty python3-docopt python3-dotenv python3-texttable python3-websocket
0 upgraded, 8 newly installed, 0 to remove and 83 not upgraded.
Need to get 297 kB of archives.
After this operation, 1589 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
```

Install kubectl

curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

chmod +x kubectl

sudo mv kubectl /usr/local/bin/

Verify

kubectl version --client

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ curl -LO "https://dl.k8s.io/rele
ase/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 55.8M  100 55.8M    0     0   320M      0 --:--:-- --:--:-- --:--:--  319M
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ chmod +x kubectl
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ sudo mv kubectl /usr/local/bin/
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ kubectl version --client
Client Version: v1.35.1
Kustomize Version: v5.7.1
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$
```

Install Minikube

curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64

sudo install minikube-linux-amd64 /usr/local/bin/minikube

Verify:

minikube version

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  128M  100  128M    0     0  36.7M      0  0:00:03  0:00:03 --:--:-- 36.8M
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ sudo install minikube-linux-amd64 /usr/local/bin/minikube
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
```

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ sudo minikube start --driver=docker
* minikube v1.38.1 on Ubuntu 24.04
* Using the docker driver based on user configuration
* The "docker" driver should not be used with root privileges. If you wish to continue as root, use --force.
* If you are running minikube within a VM, consider using --driver=none:
*   https://minikube.sigs.k8s.io/docs/reference/drivers/none/

X Exiting due to DRV_AS_ROOT: The "docker" driver should not be used with root privileges.

ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$
```

minikube start --driver=docker --cpus=2 --memory=2500

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ minikube start --driver=docker --cpus=2 --memory=2500
* minikube v1.38.1 on Ubuntu 24.04
* Using the docker driver based on user configuration
! Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
* Using Docker driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Downloading Kubernetes v1.35.1 preload ...
    > gcr.io/k8s-minikube/kicbase...:  519.55 MiB / 519.58 MiB  99.99% 104.04 M
    > preloaded-images-k8s-v18-v1...:  272.45 MiB / 272.45 MiB  100.00% 44.73 M
* Creating docker container (CPUs=2, Memory=2500MB) ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$
```

kubectl get nodes

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ kubectl get nodes
NAME       STATUS   ROLES           AGE    VERSION
minikube   Ready    control-plane   2m7s   v1.35.1
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$
```

Kubernetes Deployment

Minikube Setup

1. Start Minikube

```
minikube start
```

2. Enable Required Addons

```
minikube addons enable metrics-server
minikube addons enable ingress
```

Create Kubernetes Manifest Files

Required Manifest Files

Create following files in `k8s/` directory:

- `backend-deployment.yaml`
- `frontend-blue-deployment.yaml`
- `frontend-green-deployment.yaml`
- `frontend-service.yaml`
- `ingress.yaml`

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   51m   v1.35.1
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ minikube start
* minikube v1.38.1 on Ubuntu 24.04
* Using the docker driver based on existing profile
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Updating the running docker "minikube" container ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ minikube addons enable metrics-server
minikube addons enable ingress
* metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
  - Using image registry.k8s.io/metrics-server/metrics-server:v0.8.1
* The 'metrics-server' addon is enabled
* ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
  - Using image registry.k8s.io/ingress-nginx/controller:v1.14.3
  - Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
  - Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
* Verifying ingress addon...
* The 'ingress' addon is enabled
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$
```

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ sudo nano backend-deployment.yaml
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ sudo nano frontend-blue-deployment.yaml
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ sudo nano frontend-green-deployment.yaml
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ sudo nano frontend-service.yaml
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ sudo nano ingress.yaml
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ ls
backend-deployment.yaml  frontend-blue-deployment.yaml  frontend-green-deployment.yaml  frontend-service.yaml  ingress.yaml
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$
```

backend-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: backend:v1
          ports:
            - containerPort: 5000
```

```yaml
        env:
          - name: MONGO_URI
            value: mongodb://mongo:27017/bluegreen
        readinessProbe:
          httpGet:
            path: /health
            port: 5000
          initialDelaySeconds: 5
          periodSeconds: 10
        livenessProbe:
          httpGet:
            path: /health
            port: 5000
          initialDelaySeconds: 15
          periodSeconds: 20
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  selector:
    app: backend
  ports:
    - port: 5000
      targetPort: 5000
  type: ClusterIP
```

frontend-blue-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-blue
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
      version: blue
```

```
  template:
    metadata:
      labels:
        app: frontend
        version: blue
    spec:
      containers:
        - name: frontend
          image: frontend-blue:v2
          ports:
            - containerPort: 3100
          readinessProbe:
            httpGet:
              path: /
              port: 3100
            initialDelaySeconds: 5
            periodSeconds: 10
```

frontend-green-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-green
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
      version: green
  template:
    metadata:
      labels:
        app: frontend
        version: green
    spec:
      containers:
        - name: frontend
          image: frontend-green:v2
          imagePullPolicy: Never
          ports:
            - containerPort: 3200
```

```
        readinessProbe:
          httpGet:
            path: /
            port: 3200
          initialDelaySeconds: 5
          periodSeconds: 10
```

frontend-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  type: LoadBalancer
  selector:
    app: frontend
    version: green   # switch to green when needed
  ports:
    - port: 80
      targetPort: 3200
      nodePort: 32121
```

ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend-ingress
spec:
  rules:
    - host: bluegreen.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
```

number: 80

 # Apply all manifests
kubectl apply -f k8s/

 # Verify deployments
kubectl get deployments
kubectl get services
kubectl get pods

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ kubectl get deployments
NAME             READY   UP-TO-DATE   AVAILABLE   AGE
backend          2/2     2            2           31m
frontend-blue    2/2     2            2           47m
frontend-green   2/2     2            2           47m
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ kubectl get services
NAME               TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
backend-service    ClusterIP   10.98.60.76      <none>        5000/TCP       47m
frontend-service   NodePort    10.111.127.238   <none>        80:32121/TCP   47m
kubernetes         ClusterIP   10.96.0.1        <none>        443/TCP        112m
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ kubectl get pods
NAME                              READY   STATUS    RESTARTS   AGE
backend-5488956664-cbcwh          1/1     Running   0          31m
backend-5488956664-pbz8w          1/1     Running   0          31m
frontend-blue-d7f9b8bff-nbdtb     1/1     Running   0          35m
frontend-blue-d7f9b8bff-rfskv     1/1     Running   0          35m
frontend-green-5978c8cb9b-74crx   1/1     Running   0          2m36s
frontend-green-5978c8cb9b-d4s6q   1/1     Running   0          2m49s
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$
```

Blue-Green Switching

Switch to Green

kubectl patch service frontend-service \
  -p '{"spec":{"selector":{"version":"green"}}}'

Switch Back to Blue

kubectl patch service frontend-service \
  -p '{"spec":{"selector":{"version":"blue"}}}'

Check service endpoints:

kubectl get endpoints frontend-service

Check which pods receive traffic:

kubectl get pods -l app=frontend

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ kubectl patch service frontend-service \
  -p '{"spec":{"selector":{"version":"green"}}}'
service/frontend-service patched
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ kubectl get endpoints frontend-service
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME               ENDPOINTS                                   AGE
frontend-service   10.244.0.21:3100,10.244.0.22:3100   48m
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ kubectl get pods -l app=frontend
NAME                                READY   STATUS    RESTARTS   AGE
frontend-blue-d7f9b8bff-nbdtb       1/1     Running   0          36m
frontend-blue-d7f9b8bff-rfskv       1/1     Running   0          36m
frontend-green-5978c8cb9b-74crx     1/1     Running   0          3m54s
frontend-green-5978c8cb9b-d4s6q     1/1     Running   0          4m7s
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$
```

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ kubectl apply -f frontend-service.yaml
service/frontend-service configured
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ kubectl port-forward service/frontend-service 8080:80 --address 0.0.0.0
Forwarding from 0.0.0.0:8080 -> 3100
Handling connection for 8080
Handling connection for 8080
```

Test Blue Version

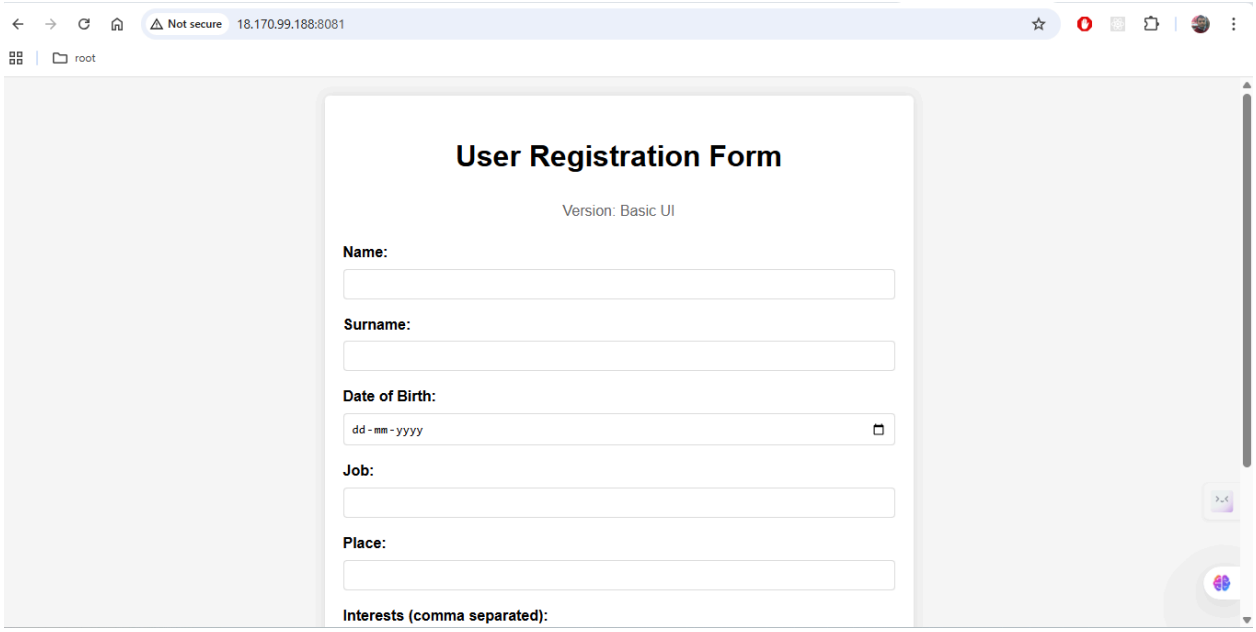
kubectl port-forward --address 0.0.0.0 pod/frontend-blue-77dfdb755b-9ldvh 8081:3100

```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ kubectl port-forward --address 0.0.0.0 pod/frontend-blue-77dfdb755b-9ldvh 8081:3100
Forwarding from 0.0.0.0:8081 -> 3100
Handling connection for 8081
Handling connection for 8081
```

Test Green Version

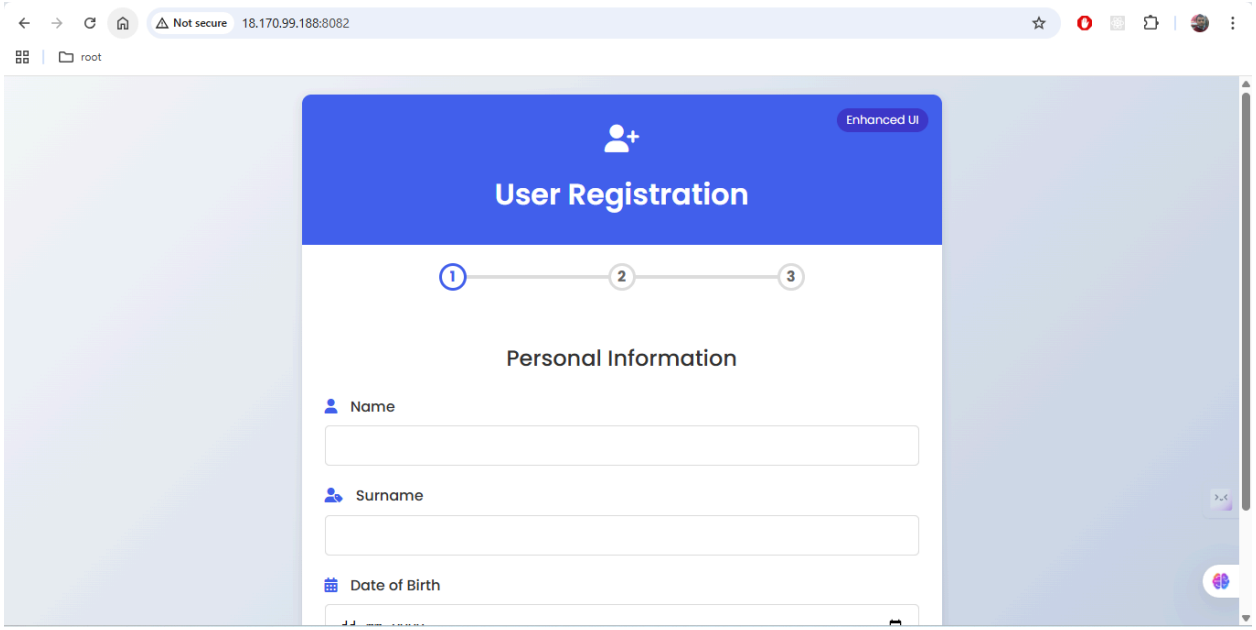
kubectl port-forward --address 0.0.0.0 pod/frontend-green-66f779bbfc-62fm8 8082:3200

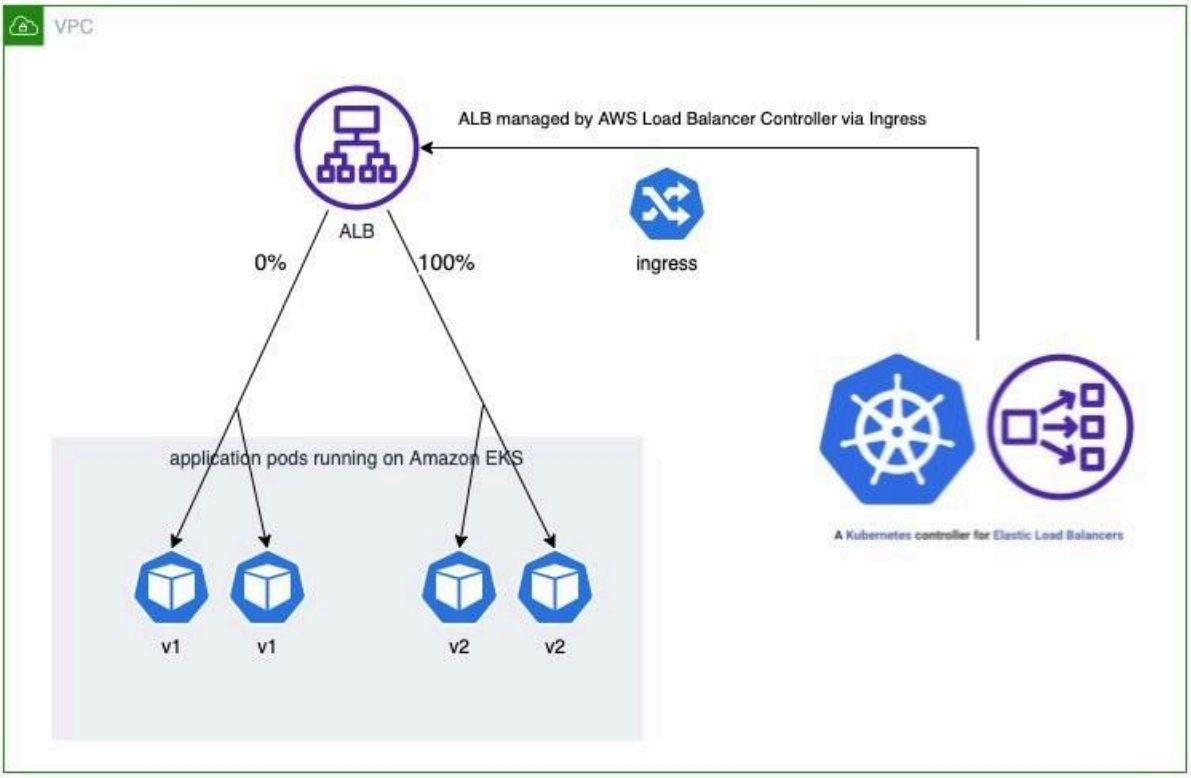
```
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment$ cd k8s/
ubuntu@ip-172-31-16-82:~/Blue-green-Deployment/k8s$ kubectl port-forward --address 0.0.0.0 pod/frontend-green-66f779bbfc-62fm8 8082:3200
Forwarding from 0.0.0.0:8082 -> 3200
Handling connection for 8082
Handling connection for 8082
```



Test Green Version

kubectl port-forward --address 0.0.0.0 pod/frontend-green-66f779bbfc-62fm8 8082:3200




VPC
ALB managed by AWS Load Balancer Controller via Ingress
ALB
0%
100%
ingress
application pods running on Amazon EKS
v1
v1
v2
v2
A Kubernetes controller for Elastic Load Balancers